

Learning as Amortized Deduction for Binary Linear Systems: Measuring the Information Ceiling, Locating Where Learning Pays, and Size Transfer under Controlled Solution Multiplicity

neuroMCTS project

September 2026

Abstract

Binary linear systems (BLS) — find $\mathbf{x} \in \{0, 1\}^n$ with $A\mathbf{x} = \mathbf{b}$ for $A \in \{0, 1\}^{m \times n}$ — arise in laser-based non-destructive testing, where a binary image must be recovered from integer projections. Deciding feasibility is NP-complete, and on the market-split-style family studied here classical solvers dominate every learned method this project has tried. Across eleven cycles, twelve learned components were tested and eleven produced null results, which had been read as evidence that the input carries no learnable signal. This report shows that reading was wrong, and replaces it with a measurement. Because the generator plants one solution $\mathbf{x}^*$ and publishes $\mathbf{b} = A\mathbf{x}^*$, every member of the solution set $\mathcal{S}$ would have produced the same $\mathbf{b}$; the posterior is therefore uniform on $\mathcal{S}$ and the exact per-variable marginal $p_j = |\{\mathbf{x} \in \mathcal{S} : x_j = 1\}|/|\mathcal{S}|$ can be computed by exhaustive enumeration. This yields a Bayes ceiling that no predictor can exceed. Four findings follow. First, the ceiling is 69.9% per-variable (93.1% on the

three most confident variables) on the hard family, and a small single-head bipartite GNN reaches within 1.9 points of it — the signal is real, bounded, and essentially exhausted, while the project’s previous multi-head network sat at the level of plain LP rounding (60.7% vs. 60.6%). Second, holding n fixed and varying m turns the ceiling into an experimental knob, and doing so separates three regimes: where the ceiling is low the model already saturates it, where the ceiling is high attaining it is NP-hard (an 18–20 point residual gap), and where it is higher still the LP relaxation alone suffices. Third, conditioning on a partial assignment is identical to reducing the instance, so the same architecture predicts *inside* the search tree; the conditional ceiling rises from 70.5% at the root to 93.4% at depth 8, and decomposing the residual shows it lies entirely on variables the prefix logically determines, 92–99% of which unit propagation fails to detect. LP-probing recovers 62–79% of them but costs one LP solve per variable; the network answers every variable in one forward pass at 1.6 ms, roughly $20\times$ cheaper, with equal or better hit rate. Learning’s role is therefore *amortization of expensive deduction*, not discovery of inaccessible information — the same role branching imitation plays in mixed-integer programming (Gasse et al., 2019). Fourth, used as branching guidance this yields a median $2.71\times$ speedup over LP-based guidance at 21×60 (sign test $p = 0.019$), a $6.2\times$ reduction in nodes, and a solve-rate increase from 90.0% to 100% within a 600 s budget; in the deployed cascade, where lattice reduction runs first, coverage of that stage falls from 30/30 at $n = 25$ to 16/30 at $n = 60$ and guidance lifts the end-to-end rate from 93.3% to 100%. Controlling $|\mathcal{S}|$ across sizes — previously confounded by a factor of ten — removes an artifact that had been misread as transfer failure: a model trained only at 10×25 is statistically indistinguishable from one trained at the target size when both are evaluated at 21×60 (15/30 instances faster, total time differing by 0.4%). All instance generators, exact-marginal labels, checkpoints and per-instance results are released with the code.

1 Introduction

1.1 The problem and where it comes from

Let $A \in \{0, 1\}^{m \times n}$ and $\mathbf{b} \in \mathbb{Z}^m$. A *binary linear system* (BLS) asks for

$$\mathbf{x}^* \in \{0, 1\}^n \quad \text{such that} \quad A\mathbf{x}^* = \mathbf{b}, \tag{1}$$

with feasible set $\mathcal{S} = \{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}\}$. Each row is a cardinality constraint $\sum_j a_{ij}x_j = b_i$: it states how many of a designated subset of cells are occupied.

The motivating application is laser-based non-destructive testing. A specimen is scanned along m directions; each scan returns an integer count of absorbing cells along its path, and the task is to reconstruct the binary occupancy image $\mathbf{x}$. Two properties of that setting shape everything below. First, $\mathcal{S}$ may contain many images consistent with the same measurements, so *uniqueness of $\mathbf{x}^*$ is not assumed*; recovering any member of $\mathcal{S}$ solves the physical problem. Second, measurement noise (dust, calibration drift) can perturb $\mathbf{b}$ so that $\mathcal{S} = \emptyset$, and reporting infeasibility is as operationally important as returning a solution.

Deciding whether $\mathcal{S} = \emptyset$ is NP-complete, and the instances used here are deliberately drawn from a hard regime (Section 4.1) inspired by market-split problems (Cornuéjols and Dawande, 1999; Aardal et al., 2000b), which are standard stress tests for both integer programming and lattice methods.

1.2 The question this report answers

This project has spent eleven development cycles attaching learned components to a symbolic solving pipeline: an AlphaZero-style policy over a Monte-Carlo tree search, an imitation-trained branching policy, a learned scheduler for lattice restarts, ten feasibility classifiers, a permutation policy, and a variable-fixing policy. Eleven of twelve produced null results. The standing interpretation was that $(A, \mathbf{b})$ simply carries no signal a network can use.

That interpretation was never tested. It is a claim about the information content of the input, and information content is measurable. The central observation of this report is that for this instance family it is measurable *exactly*:

The generator samples $\mathbf{x}^*$ and publishes $\mathbf{b} = A\mathbf{x}^*$. Every $\mathbf{x} \in \mathcal{S}$ would have produced the identical $\mathbf{b}$. Nothing in the input distinguishes them, so the posterior over “which solution was planted” is uniform on $\mathcal{S}$, and the Bayes-optimal per-variable prediction is $\arg \max$ of

$$p_j = \Pr[x_j = 1 \mid A, \mathbf{b}] = \frac{|\{\mathbf{x} \in \mathcal{S} : x_j = 1\}|}{|\mathcal{S}|}, \quad (2)$$

which is right a fraction $\max(p_j, 1 - p_j)$ of the time. Enumerating $\mathcal{S}$ therefore gives an exact upper bound on what *any* predictor can achieve.

Enumeration is feasible at $n \leq 60$ with a constraint solver, which makes the ceiling an experimental quantity rather than a theoretical one. Everything in this report follows from being able to measure it.

1.3 Contributions

1. **A measured information ceiling.** The Bayes-optimal per-variable accuracy on the hard family is 69.9% (Section 4.2). A small single-head network reaches 68.6%, within 1.9 points, while the project’s previous multi-head network reached 60.7% — indistinguishable from LP rounding at 60.6%. The signal is real, small, and now essentially exhausted: past nulls reflected model and task mismatch at least as much as absent signal.
2. **The ceiling as a controlled variable.** Holding n fixed and raising m shrinks $|\mathcal{S}|$ and lifts the ceiling from 70.5% to 100%. Sweeping it exposes three regimes and shows there is no operating point at which learning wins by a wide margin: where the ceiling

is low it is already attained; where it is high, attaining it is NP-hard; where it is higher still, the LP relaxation suffices (Section 4.3).

3. **Learning as amortized deduction.** Conditioning on a partial assignment is identical to reducing the instance, so the same architecture predicts inside the tree. The conditional ceiling rises to 93.4% by depth 8; the residual gap lies *entirely* on variables the prefix logically forces, of which unit propagation detects only 0.9–7.5%. LP-probing recovers 62–79% at one LP solve per variable (≈ 33 ms per node); the network answers all variables in 1.6 ms with equal or better hit rate — a 19–23 $\times$ cost advantage (Sections 4.4–4.5).
4. **End-to-end gains, and transfer under controlled $|\mathcal{S}|$.** As branching guidance the model is 2.71 $\times$ faster than LP guidance at 21×60 ($p = 0.019$), uses 6.2 $\times$ fewer nodes, and raises the solve rate from 90.0% to 100%; in the deployed cascade it lifts 93.3% to 100%. With $|\mathcal{S}|$ matched across sizes, a model trained only at 10×25 is statistically indistinguishable from one trained at the target size (Sections 4.6–4.7).
5. **A negative-result map with causes.** Twelve learned components are tabulated with the mechanism behind each outcome, including two of our own misreadings that measurement corrected (Section 4.8).

2 Background and Related Work

This section is written to be self-contained: a reader who has not seen the cited work should be able to follow the rest of the report without consulting it.

2.1 Why these instances are hard

For a general 0/1 matrix A , deciding $\mathcal{S} \neq \emptyset$ is NP-complete, so worst-case hardness is not in question. What matters practically is whether *typical* instances are hard, and that depends

on how they are generated.

The family used here follows the market-split construction of Cornuéjols and Dawande (1999). In the original, m agents each hold a budget and n items must be split so every agent’s budget is met exactly; with $n \approx 10(m - 1)$ these become notoriously difficult for branch-and-bound despite their small size. The reason is the *integrality gap*: the linear relaxation

$$\tilde{\mathcal{S}} = \{\mathbf{x} \in [0, 1]^n : A\mathbf{x} = \mathbf{b}\} \quad (3)$$

is a large, well-populated polytope whose vertices are mostly fractional. Branch-and-bound must explore exponentially many nodes because the relaxation gives almost no guidance about which variables to fix. This report exploits the same mechanism deliberately: our generator maximizes *vertex spread*, the mean pairwise distance among LP vertices obtained from random objective directions, which enlarges $\tilde{\mathcal{S}}$ and weakens the LP signal by construction (Section 4.1).

2.2 Classical solvers, and what makes CP-SAT strong

Three classical approaches are relevant.

Mixed-integer programming (MIP). Gurobi and SCIP solve (1) by branch-and-bound over the LP relaxation, strengthened by presolve and cutting planes. Their leverage comes from bounding: a node whose relaxation is infeasible is pruned. When the relaxation is uninformative — exactly the market-split regime — that leverage weakens.

Constraint programming with clause learning (CP-SAT). Google OR-Tools’ CP-SAT is consistently the strongest baseline in our measurements, and it is worth stating precisely why, because the reason shapes this report’s thesis. CP-SAT is not a plain backtracking search. It combines:

- **Constraint propagation**, which *deduces* forced values from a partial assignment

rather than guessing them;

- **Conflict-driven clause learning (CDCL)** (Marques-Silva and Sakallah, 1999; Moskewicz et al., 2001), which, on reaching a contradiction, analyses the implication graph to derive a new clause that is valid everywhere and prunes an exponentially large region of the remaining space;
- **Lazy clause generation** (Ohrimenko et al., 2009), which explains propagations in clausal form so that CP-level reasoning feeds the SAT-level learning;
- **Restarts with clause retention**, and an LP relaxation supplying cutting planes.

The key point is the source of information. A one-shot predictor consumes only what is present in $(A, \mathbf{b})$. A search *manufactures* information as it descends: $\Pr[x_j = 1 \mid A, \mathbf{b}]$ may be exactly 0.5 while $\Pr[x_j = 1 \mid A, \mathbf{b}, x_1 = 1, x_5 = 0]$ is 0 or 1. Structure invisible at the root becomes deducible after commitments are made. This is why CP-SAT can solve in milliseconds instances whose root marginals are uninformative, and it is also why Section 4.4 looks *inside* the tree rather than at the root.

To confirm that clause learning is the operative ingredient rather than constraint propagation alone, we ran a plain CP solver (OR-Tools’ `pywrapcp`, no CDCL) on the same instances: it solves 12/12 at 10×25 but 0/12 at 20×50 , where CP-SAT solves 100%.

Lattice reduction. An alternative attack embeds (1) in a lattice and looks for short vectors. LLL (Lenstra et al., 1982) produces a reduced basis in polynomial time; BKZ (Schnorr and Euchner, 1994) generalizes it with a block size β that trades running time for basis quality. Aardal et al. (2000a,b) give an embedding (AHL) in which a 0/1 solution of a market-split system appears *directly* as a short vector of a suitably constructed lattice, so a reduction that finds it yields a verified solution with no search. In our pipeline AHL is the single most productive component, and its block size must *decrease* as instances grow: block 40 costs over 200s per attempt at $n = 100$ while block 20 costs 2.4s with better coverage.

2.3 Learning for combinatorial optimization

Where learning has succeeded. The clearest success is *imitation of an expensive expert*. Strong branching — tentatively solving both children’s relaxations to score a branching candidate — produces small search trees but is far too slow to run at every node. Gasse et al. (2019) train a bipartite GNN to imitate it and recover most of the benefit at a small fraction of the cost. Khalil et al. (2016) make the same move with learned ranking. The gain is *not* new information: strong branching already knew the answer. The gain is that the knowledge becomes cheap enough to use everywhere. This report argues the same structure applies here, and measures it.

Where learning has failed, and why. Attempts to have a network decide satisfiability directly have a weaker record. NeuroSAT (Selsam et al., 2019) classifies satisfiability by message passing over a literal–clause graph and succeeds on small random instances, but G4SATBench (Li et al., 2023) reports 15–25 point accuracy collapse under size transfer. More fundamentally, Chen et al. (2023) prove that standard message-passing GNNs cannot separate certain feasible/infeasible MIP pairs at all: the pairs are indistinguishable in the Weisfeiler–Lehman hierarchy that bounds such networks (Xu et al., 2019). Random node features are the literature’s prescribed remedy; we tested them at dimensions 8 and 32 and obtained AUC 0.4997–0.5003, i.e. chance.

The gap this report addresses. Both literatures report *outcomes* — this worked, that did not. Neither establishes how much signal the input contains, so a null result cannot be attributed between “no information” and “information present but not extracted.” For planted-solution instances that attribution is computable, and making it is this report’s methodological contribution.

2.4 This project’s own prior cycles

The pipeline this report builds on was developed over cycles v4–v12 and is summarized here because later sections modify it. Its symbolic stages are: constraint propagation with probing; an LP-relaxation infeasibility rule; a *kernel pump* that alternately projects between the affine solution space and the unit hypercube; AHL lattice reduction; and a complete backtracking search. Its learned stages were a multi-head bipartite GNN (selection, assignment, value, feasibility) driving MCTS and branching.

Two prior measurements matter here. First, an instance-by-instance attribution found the learned stages contributed 7.6 points of solution accuracy at 10×25 , 0.4 points at 20×50 , and *exactly zero* at 40×100 and 60×150 — AHL and propagation accounted for everything. Second, a matched-budget study found returns to additional search compute sharply diminishing ($3.8\times$ time bought 4 points), locating the binding constraint in search technology rather than model capacity. Sections 3 onward retire the zero-contribution stages and rebuild the learned component around a different objective.

3 Proposed Method

3.1 Overview

The method has three parts: a measurement procedure that establishes what is attainable (§3.2), a predictor trained against exact posteriors rather than planted samples (§3.3–3.5), and an integration that uses the predictor where its cost profile makes it worth using (§3.6–3.7).

3.2 Measuring the ceiling

Given $(A, \mathbf{b})$ we enumerate $\mathcal{S}$ exhaustively with CP-SAT in all-solutions mode and compute the exact marginals (2). The Bayes ceiling on per-variable accuracy is

$$\text{ceil}(A, \mathbf{b}) = \frac{1}{n} \sum_{j=1}^n \max(p_j, 1 - p_j). \quad (4)$$

No deterministic predictor $f(A, \mathbf{b})$ can exceed this: instances sharing $(A, \mathbf{b})$ are indistinguishable to f , so the best available answer per variable is the majority value among surviving solutions.

A correctness condition that is easy to get wrong. An enumeration is admissible only if it *completed*. CP-SAT returns `OPTIMAL` when the search is exhausted, `INFEASIBLE` when there is nothing to enumerate, and `FEASIBLE` when a time limit interrupted a search that had already collected solutions. Treating `FEASIBLE` as completion silently substitutes a truncated $\mathcal{S}$, which biases every marginal computed from it. We hit this bug: it made $n = 100$ appear to have measurable $|\mathcal{S}|$ (medians of 688 and 2) when in fact no enumeration at $n = 100$ finishes within two minutes. The symptom was that mean solve time equalled the time limit exactly. Only `OPTIMAL` and `INFEASIBLE` are accepted, and instances that time out are discarded rather than averaged in.

3.3 Training target: exact posteriors, not planted samples

Every previous cycle trained against the planted solution, i.e. used $x_j^* \in \{0, 1\}$ as the label. When $|\mathcal{S}| > 1$ this is a *sample from* the posterior, not the posterior: at 10×25 the median $|\mathcal{S}|$ is 23, and the Bayes-optimal prediction agrees with $\mathbf{x}^*$ only 66% of the time. Training on $\mathbf{x}^*$ is therefore training with roughly 34% label noise, and because each instance is visited

about once, that noise never averages out. We instead train against p_j directly, minimizing

$$\mathcal{L} = -\frac{1}{n} \sum_{j=1}^n \left[p_j \log \hat{p}_j + (1 - p_j) \log(1 - \hat{p}_j) \right], \quad (5)$$

the cross-entropy to a Bernoulli(p_j) target, which is minimized exactly at $\hat{p}_j = p_j$.

3.4 Conditioning equals reduction

The step that makes in-tree prediction cheap to implement is an identity. Fix a set F of variables to values $\mathbf{v}$ and let K be the remaining indices. Then

$$\{\mathbf{x} \in \{0, 1\}^n : A\mathbf{x} = \mathbf{b}, \mathbf{x}_F = \mathbf{v}\} \cong \{\mathbf{y} \in \{0, 1\}^{|K|} : A_K \mathbf{y} = \mathbf{b} - A_F \mathbf{v}\}, \quad (6)$$

so the conditional marginals at a depth- d node are exactly the ordinary marginals of a smaller BLS instance. One architecture, one training procedure and one label-generation routine therefore serve both the root and the interior of the tree; a depth- d state is stored simply as a smaller $(A', \mathbf{b}')$ pair with its own exact marginals.

Two details keep the resulting data honest. States are built by taking a prefix from a *real* solution, so every training state is reachable — a prefix consistent with no solution is a dead node that propagation would have pruned, not a state worth learning on. Rows that become all-zero after substitution are dropped: they impose no constraint, and SCIP rejects an empty constraint outright.

3.5 Architecture

The instance is a bipartite graph with n variable nodes and m constraint nodes, with an edge wherever $a_{ij} = 1$. Variable features are the LP relaxation value $\tilde{x}_j$, the column degree $\deg(j)/m$, and the fractionality $|\tilde{x}_j - \text{round}(\tilde{x}_j)|$. Constraint features are tightness b_i/rowsum_i , scaled row size rowsum_i/n , and LP residual $(b_i - A_i \tilde{\mathbf{x}})/\text{rowsum}_i$. Both are

embedded to 64 dimensions and refined by four rounds of

$$\mathbf{h}^{\text{con}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{var}} \rightarrow \mathbf{h}^{\text{con}})) + \mathbf{h}^{\text{con}}), \quad \mathbf{h}^{\text{var}} \leftarrow \text{LN}(\text{ELU}(\text{GAT}(\mathbf{h}^{\text{con}} \rightarrow \mathbf{h}^{\text{var}})) + \mathbf{h}^{\text{var}}), \quad (7)$$

using graph attention (Veličković et al., 2018) so the network can weight constraints by relevance. A two-layer head emits one logit per variable, giving $\hat{p}_j = \sigma(z_j)$. We call this network **MarginalNet**.

Because the parameters depend only on feature dimensions and not on m or n , one set of weights applies to any instance size — the structural precondition for the transfer experiments of Section 4.7.

Deliberate contrast with the prior network. The project’s earlier model had hidden width 256, eight layers and four heads (selection, assignment, value, feasibility) serving MCTS and DFS. MarginalNet is 64-wide, four layers, one head. It is roughly $84\times$ smaller and, as Section 4.2 shows, substantially more accurate at this task — the multi-head model performed at the level of LP rounding. We retired the heads along with the stages they served, which had been measured at zero contribution.

3.6 Using the predictor: guidance, not commitment

MarginalNet outputs a probability, not a proof, so it is used only where a wrong answer costs time rather than correctness. Concretely, branching selects the variable $j^* = \arg \max_j |\hat{p}_j - \frac{1}{2}|$ and tries value $[\hat{p}_{j^*} \geq \frac{1}{2}]$ first; the search remains complete, and a mistake costs backtracking. Variables are *fixed* only by sound deduction — propagation, or LP-probing, which fixes x_j when assigning the opposite value renders even the relaxation infeasible. This division follows directly from the cost–soundness measurements of Section 4.5.

The search loop itself is standard depth-first backtracking: check the free bound condition $0 \leq \mathbf{b} \leq A\mathbf{1}$, propagate, fold any newly determined variables into a reduced instance via (6),

then consult the guidance function for the next branching decision. Four guidance strategies are compared in Section 4.6: RANDOM, LP, LP-PROBE, and MODEL.

3.7 The deployed cascade

In deployment the guided search is not used alone. The pipeline runs AHL lattice reduction first and invokes the search only on instances AHL fails to close:

$$\text{propagation} \rightarrow \text{LP rule} \rightarrow \text{kernel pump} \rightarrow \text{AHL/BKZ} \rightarrow \text{guided search}. \quad (8)$$

AHL is oblivious to the guidance strategy, so every arm receives *exactly the same* residual subset; conditioning on it describes the algorithm rather than selecting favourable instances. We report both this configuration (AHL ON) and the ablation with AHL removed (AHL OFF) at every size. Reporting only the ablation, as an earlier cycle did, is what produced the misreading corrected in Section 4.7: removing AHL makes 40×100 unsolvable for *every* arm, and a collapse common to all arms had been attributed to the learned one.

4 Experiments

4.1 Instances, and controlling solution multiplicity

Instances are generated by `gen_hard_feasible( $m, n, \text{rng}, K$ )`. For a random 0/1 matrix A of density $1/2$, it plants $K = 20$ candidate solutions, computes $\mathbf{b} = A\mathbf{x}$ for each, and keeps the one maximizing *vertex spread* — the mean pairwise L_2 distance among LP vertices obtained from random objective directions. This enlarges the relaxation polytope directly, which is the mechanism that makes market-split instances hard (§2.1). Uniqueness is not enforced.

$|\mathcal{S}|$ as a controlled variable. Solution multiplicity changes the *nature* of the task, not merely its difficulty: when $|\mathcal{S}| = 1$ every variable is logically determined and predicting it well amounts to solving the instance, whereas when $|\mathcal{S}| = 23$ many variables are genuinely undetermined and the task is to estimate a smooth posterior. Comparing sizes without controlling $|\mathcal{S}|$ therefore confounds size with task identity. Since $|\mathcal{S}|$ falls as m rises at fixed n , we select m per size to match $|\mathcal{S}|$:

Size	m	m/n	median $ \mathcal{S} $	root ceiling	conditional ceiling
10×25	10	0.40	23	70.4%	87.0%
18×50	18	0.36	19	71.3%	87.2%
21×60	21	0.35	10	—	—

Table 1: Sizes with matched solution multiplicity. Ceilings agree to within one point, so size is the only variable that changes. Previously the same comparison ran at conditional ceilings of 87.4% vs. 98.6%.

What this costs, and where it stops. Matching $|\mathcal{S}|$ forces m/n to vary ($0.40 \rightarrow 0.35$); at fixed n the two cannot be held simultaneously, and we prioritized $|\mathcal{S}|$ because it determines task identity. The control also has a hard limit. At $n = 60$ the median $|\mathcal{S}|$ steps sharply with m — 42 at $m = 20$, 12 at $m = 21$, 2 at $m = 22$ — so $m = 21$ is the best integer available, and the realized test set has median 10 with 27/30 verified. At $n = 70$ no enumeration completed at the m giving the right multiplicity (0/10 at $m = 23$). At $n = 100$ the failure is more basic: CP-SAT finds *zero* solutions within 20s at $m = 28, 32, 36, 40$ despite a planted one. At that size the bottleneck is not guidance quality but finding any solution at all, so no guidance comparison is meaningful there — which retrospectively explains why all arms collapsed to 8–12% at 40×100 in earlier work.

4.2 The Bayes ceiling and how close models come to it

We enumerated $\mathcal{S}$ for 10×25 instances (median $|\mathcal{S}| = 24$, under 1 s each), trained Marginal-Net on 1,500 instances from a disjoint pool, and evaluated on 400 held-out instances with

complete enumeration.

Predictor	all-variable acc.	top-3	top-5	L_1 to true marginal
Bayes ceiling	69.9%	93.1%	90.5%	0.0000
MarginalNet (hard labels)	68.6%	87.4%	85.2%	0.1261
MarginalNet (soft labels)	67.7%	87.2%	85.3%	0.0977
Prior multi-head GNN	60.7%	72.0%	69.5%	0.1813
LP relaxation	60.6%	64.5%	64.1%	0.3213

Table 2: Held-out 10×25 , 400 instances with exhaustively enumerated $\mathcal{S}$.

Three readings. (i) *The signal is real but small*: 69.9% is well above chance, refuting “no information,” yet far from usable on its own — at $0.85^5 \approx 44\%$, even top-5 accuracy does not make a five-variable joint commitment reliable. (ii) *A small dedicated model nearly exhausts it*: MarginalNet is within 1.9 points overall and 5.7 on top-3, while the prior multi-head network is statistically indistinguishable from LP rounding (60.7% vs. 60.6%) — it had learned little beyond copying its LP input feature. This reframes several past nulls as model and task mismatch rather than absent signal. (iii) *The label-noise hypothesis is only partly right*: soft labels clearly improve posterior estimation (L_1 0.1261 $\rightarrow$ 0.0977, -22%) but leave classification accuracy unchanged, so noisy labels were not the principal cause of past failures.

4.3 Sweeping the ceiling: three regimes, no wide-margin regime

Holding $n = 25$ and raising m shrinks $|\mathcal{S}|$ and lifts the ceiling. Training the identical model at each m asks whether accuracy follows.

m	median $ \mathcal{S} $	unique	ceiling	MarginalNet	gap	LP
10	24	0%	70.5%	68.5%	1.9	61.2%
14	1	76%	96.0%	76.5%	19.6	71.9%
16	1	97%	99.2%	81.2%	17.9	78.4%
20	1	100%	100.0%	99.5%	0.5	99.5%

Table 3: $n = 25$, m varied. The gap column is what separates the regimes.

At $m = 10$ — our hard family — the model saturates the ceiling, so the binding constraint is the ceiling itself, which is low precisely because we maximize polytope size. At $m = 14$ –16 the ceiling is high but a 18–20 point gap opens: a high ceiling here means the solution is essentially unique, so attaining it is equivalent to solving an NP-hard instance, and the gap is that hardness made visible. At $m = 20$ the LP relaxation alone reaches 99.5% and there is nothing to learn. *There is no operating point where learning wins by a wide margin* — where the ceiling is reachable it is already reached, and where it is high it is computationally out of reach. Note also that the prior network tracks LP at every m (60.9 vs. 61.2; 71.9 vs. 71.9; 78.5 vs. 78.4), confirming across four regimes that it never learned beyond its LP input.

4.4 Inside the tree: where the information actually is

The root ceiling of 70.5% bounds one-shot prediction, not search. Using (6) we measured, at each depth d along a prefix taken from a real solution, both the conditional ceiling and how much unit propagation already forces.

depth	surviving $ \mathcal{S} $	conditional ceiling	forced by propagation	MarginalNet
0	25.2	70.5%	0.0%	67.9%
4	4.1	82.2%	0.4%	73.2%
7	1.9	92.6%	4.1%	80.4%
8	1.6	93.4%	8.2%	85.0%
12	1.1	98.6%	56.7%	97.4%
15	1.0	99.7%	94.6%	—

Table 4: Conditional information versus depth, $n = 25$, 150 instances, LP-confidence variable order.

The ceiling rises steeply — 70.5% $\rightarrow$ 93.4% by depth 8 — while propagation still forces only 8.2% of remaining variables. Training on in-tree states beats the root-only model by 5.3–11.5 points over depths 6–10, confirming that in-tree learning is worthwhile in itself.

A correction to our own framing. We initially scored the “learning window” as $(\text{ceiling} - 0.5) \times (1 - \text{fraction forced})$, treating propagation as the only competitor. That was wrong:

the real competitor is the LP relaxation *re-solved on the reduced instance*, which strengthens rapidly with depth (60.6% $\rightarrow$ 97.2%) because substitution tightens the polytope. Measured against it, the model’s net advantage decays from +7.3 points at depth 0 to +0.2 at depth 12. The genuine residual is at depths 6–7, where the gap to the ceiling is largest (10.8–12.2 points) and the model still leads LP by 3.8–4.1. The methodological lesson is that the choice of baseline determines the conclusion; omitting one competitor inflated the apparent opportunity.

4.5 Decomposing the residual, and the cost axis

At depth 6–8 roughly two solutions survive, so each free variable is either FORCED (all survivors agree; $p_j \in \{0, 1\}$) or FREE (survivors disagree). The ceiling is attainable exactly on FORCED variables, so any shortfall must live there.

depth	% FORCED	model on FORCED	prop. detects	LP-probe detects	model on FREE
5	61.5%	90.4%	0.9%	62.2%	57.9%
6	72.1%	89.2%	1.6%	68.1%	52.3%
7	83.0%	87.9%	4.1%	75.0%	52.9%
8	89.0%	89.2%	7.5%	79.4%	59.7%

Table 5: Residual decomposition. “Detects” columns are recall on the ground-truth FORCED set.

The gap is entirely on FORCED variables ($0.83 \times 12.1 \approx 10$ points at depth 7, matching the measured 12.2); FREE performance near 50% is correct behaviour, not a defect. The striking number is that unit propagation detects only 0.9–7.5% of logically determined variables. LP-probing — assigning the opposite value and testing whether the relaxation becomes infeasible — recovers 62–79%, and together with LP integrality 82.8–92.0%.

We first concluded from this that classical reasoning matches the model and learning is unnecessary. That conclusion compared different quantities: probing’s number is *detection recall* (when it fires it constitutes a proof; otherwise it abstains), while the model’s is *guess accuracy* over all variables. Adding the missing axis settles it:

depth	MarginalNet time	acc. on FORCED	LP-probe time	detects	speed ratio
5	1.59 ms	89.9%	36.85 ms	60.5%	23.1×
6	1.61 ms	91.3%	35.30 ms	68.8%	22.0×
7	1.58 ms	88.2%	32.74 ms	77.9%	20.8×
8	1.59 ms	88.5%	30.99 ms	79.8%	19.5×

Table 6: Per-node cost for a whole-instance verdict, single-threaded. Probing pays one LP solve *per variable*; the network answers all variables in one forward pass.

The model is roughly 20× cheaper with equal or better hit rate. This fixes the division of labour: for *guidance*, where a wrong answer costs backtracking, the network is strictly preferable; for *sound fixing*, where a wrong answer costs correctness, probing is required and the network cannot substitute. Learning’s contribution is thus amortization of deduction that is otherwise too expensive to run at every node — structurally the same role as branching imitation in MIP (Gasse et al., 2019).

4.6 Guided search

We compare four guidance strategies under a matched wall-clock budget, with AHL disabled so that branching quality is isolated. Each arm gets 30 instances.

Size (budget)	guidance	solve rate	median nodes	median time
10 × 25 (60 s)	RANDOM	100%	1,514	0.07 s
	LP	100%	66	0.12 s
	LP-PROBE	100%	12	0.79 s
	MODEL	100%	22	0.08 s
20 × 50 (300 s)	RANDOM	0%	—	—
	LP	100%	8,876	24.93 s
	LP-PROBE	100%	173	28.14 s
	MODEL	100%	1,676	6.37 s

Table 7: Guidance comparison, AHL disabled.

The trade-off is explicit. LP-PROBE yields by far the smallest trees (51× fewer nodes than LP at 20 × 50) but is the slowest in wall-clock, processing only 6 nodes/s against 376 for LP. MODEL sits at the useful point: 5.3× fewer nodes than LP at a per-node cost below LP’s,

giving $3.9\times$ lower median time. Node count and wall-clock rank these methods in *opposite* orders, so evaluating branching heuristics by tree size alone — a common convention — inverts the conclusion here.

4.7 Size transfer with $|\mathcal{S}|$ controlled

The full factorial crosses four transfer conditions with the AHL switch. Conditions A–C use a model trained only at 10×25 ; condition D and the $n = 50$ reference use models trained at the target size, isolating transfer loss.

Condition	AHL	arm	solve rate	by AHL	median search	median nodes
A: $25 \rightarrow 25$	off	LP / MODEL	100% / 100%	0	0.08 / 0.08 s	47 / 23
	on	both	100%	30/30	—	—
B: $25 \rightarrow 50$	off	LP	100%	0	15.64 s	6,208
	off	MODEL	100%	0	2.84 s	833
	on	both	100%	24/30	0.03 s	—
Ref: $50 \rightarrow 50$	off	MODEL	100%	0	3.42 s	1,001
C: $25 \rightarrow 60$	off	LP	90.0%	0	190.76 s	69,015
	off	MODEL	100%	0	47.28 s	12,010
	on	LP	93.3%	16/30	12/14 residual	—
	on	MODEL	100%	16/30	14/14 residual	—
	on	MODEL	100%	17/30	13/13 residual	—
D: $50 \rightarrow 60$	off	MODEL	100%	0	54.75 s	14,108
	on	MODEL	100%	17/30	13/13 residual	—

Table 8: Conditions A–D $\times$ {AHL off, AHL on}, 30 instances each, 600 s budget at $n = 60$.

Guidance quality scales from speed to solve rate. At $n = 50$ the model is faster on 22/30 instances (median $3.59\times$) with no solve-rate difference. At $n = 60$ it is faster on 20/27 commonly solved instances (median $2.71\times$, sign test $p = 0.019$), uses $6.2\times$ fewer nodes, and cuts total time from 6,227 s to 2,421 s — and it additionally solves three instances on which LP exhausted its budget, closing them in 41, 230 and 232 s. We report the two results with different confidence: the speed advantage is significant, whereas the solve-rate advantage is perfectly consistent in direction (3–0; LP wins no instance) but underpowered, since a 3–0 split gives McNemar $p = 0.250$ at best and significance would require 5–0.

Transfer loss is not measurable. On the same $n = 60$ test set, the model trained only at 10×25 and the one trained at 18×50 are indistinguishable: the former is faster on exactly 15/30 instances, median ratio $0.92\times$, and the totals differ by 0.4% (2,924.9 s vs. 2,914.5 s), with both solving 30/30. Training at $2.4\times$ smaller scale costs nothing measurable. Root-level prediction accuracy tells the same story: the 10×25 model scores 67.8%, 69.5% and 69.4% at 10×25 , 20×50 and 40×100 , holding its margin over LP (+11.0, +7.2, +4.7) at every size.

Why an earlier cycle read this as failure. A previous cycle concluded transfer had failed at 40×100 . Two confounds produced that reading: the $|\mathcal{S}|$ mismatch corrected here, and an ablation artifact — with AHL removed, *no* arm solves 40×100 (all at 8–12%), because plain DFS cannot handle $n = 100$; the real pipeline solves 54–62% there using AHL. A collapse common to every arm had been attributed to the learned one. Running both AHL settings at every size, as in Table 8, is what makes the comparison interpretable.

Where the deployed gain comes from. AHL coverage falls with size — 30/30 at $n = 25$, 24/30 at $n = 50$, 16/30 at $n = 60$. This is what creates room for guidance: at $n = 60$ fourteen instances reach the search, where MODEL closes 14/14 and LP 12/14, lifting the end-to-end rate from 93.3% to 100%. Earlier cycles found learned components contributing zero end-to-end precisely because lattice reduction absorbed nearly every instance before search began; $n = 60$ is the first size where that ceases to hold while search still functions.

4.8 Negative results and their causes

Two of our own readings were themselves corrected by measurement, and we record them because they are the kind of error the methodology is meant to catch: treating propagation as the only competitor when scoring the learning window (§4.4), and comparing probing’s detection recall against the model’s guess accuracy as if they were the same quantity (§4.5).

Component	Outcome	Diagnosed cause
Self-play MCTS policy	9.6 $\rightarrow$ 5.9%	open-loop training degrades a pre-trained search policy
Expert imitation (branching)	8.5 $\rightarrow$ 5.5%	fail-first vs. succeed-first asymmetry; helps refutation, harms search
Learned AHL restart scheduler	rejected	feature rank-AUC 0.54–0.58, too weak to schedule on
Feasibility classifiers ($\times 10$)	AUC 0.4997–0.5003	representation provably uninformative; 96.8% of near-miss pairs have bit-identical GS profiles (Chen et al., 2023)
Random node features	no change	prescribed remedy for WL limits does not apply here
AHL block-size policy	noise-level	fixing the default gives +14.4 points; per-instance choice adds nothing
CP-SAT parameter policy	no signal	oracle over 6 configs equals the default
Multi-size fine-tuning (109 ep.)	$\Delta = 0$	the tuned stages contributed 0% to begin with
AHL permutation policy (RL)	$\Delta = -0.0005$	signal exists (8/10 instances permutation-dependent) but is not learnable from observable features
Variable-fixing policy (RL)	not completable	all-or-nothing reward over k decisions destroys credit assignment

Table 9: Twelve learned components; eleven null. Causes are measured, not conjectured.

5 Discussion

5.1 What the measurements support

The defensible claim is narrow and, we think, more useful than the one the project began with:

Learning does not supply information that classical reasoning cannot reach. It makes deduction that is otherwise too expensive to run at every node cheap enough to use, and this capability transfers across instance size without measurable loss once solution multiplicity is controlled.

Each clause is measured. “Does not supply new information”: the residual gap lies entirely on logically FORCED variables, of which classical probing recovers 82–92% (§4.5). “Too expensive to run at every node”: LP-probing costs 31–37 ms per node against the

network’s 1.6 ms (Table 6). “Transfers without measurable loss”: 15/30 and 0.4% total-time difference (§4.7).

The weaker claim — that learning finds structure classical methods miss — is contradicted by our own data and should not be made.

5.2 Limitations

The ceiling is low, and we lowered it. Maximizing vertex spread is what makes these instances hard for CP-SAT, and it is also what drives the root ceiling to 69.9%. An easier family would offer more signal but would not be the target regime.

The useful size window is narrow. Below it ($n \leq 25$) lattice reduction closes everything and guidance is irrelevant; above it ($n = 100$) search fails outright, CP-SAT finding zero solutions in 20 s. Our demonstration lives at $n = 60$, and we have not shown the window extends further.

Enumeration bounds the method, not just the evaluation. Exact marginal labels require enumerating $\mathcal{S}$, which fails beyond $n \approx 60$ on this family. Transfer means models trained on enumerable sizes can be *applied* to larger ones, but training data cannot be generated there.

Statistical power. The solve-rate advantage at $n = 60$ is 3–0 in direction with $p = 0.250$; only the speed advantage ($p = 0.019$) is significant. Larger test sets are needed to settle the former.

$|\mathcal{S}|$ control is approximate. Medians are 23, 19 and 10; at $n = 60$ multiplicity steps $42 \rightarrow 12 \rightarrow 2$ across consecutive integer m , so finer matching is not available. Three of thirty test instances have unverified $|\mathcal{S}|$. Matching $|\mathcal{S}|$ also forces m/n to drift from 0.40 to 0.35.

Single application domain. All results concern one instance family from one application. Whether the amortization framing transfers to other constraint families is untested.

5.3 Future work

Three directions follow from the measurements rather than from speculation. (i) *Approximate marginals beyond enumeration*: if p_j could be estimated by sampling (e.g. solution sampling or belief propagation) with usable fidelity, training data could be generated where enumeration fails. (ii) *Learning to imitate LP-probing directly*: Table 6 shows the network already matches probing’s hit rate at 1/20 the cost without being trained to imitate it; explicit distillation from probing decisions is the natural next step, and is precisely the learn2branch recipe. (iii) *Replacing the pipeline’s propagator*: the measurement that unit propagation detects under 8% of forced variables while LP-probing detects 62–79% is an improvement available immediately and independent of learning.

6 Conclusion

Eleven cycles of null results on binary linear systems had been read as evidence that the input carries no learnable signal. Because these instances have planted solutions, that reading was testable, and it was wrong. Enumerating the solution set gives the exact posterior and therefore an exact Bayes ceiling: 69.9% per-variable on the hard family, 93.1% on the three most confident variables. A small single-head network comes within 1.9 points, whereas the project’s previous multi-head network performed at the level of LP rounding — so several earlier nulls reflected model and task mismatch rather than absent information.

Turning the ceiling into a controlled variable shows why the signal is nonetheless hard to profit from: where the ceiling is low it is already attained, where it is high attaining it is NP-hard, and where it is higher still the LP relaxation suffices. The productive question is therefore not how much a network knows but how cheaply it knows it. Inside the search

tree the conditional ceiling reaches 93.4% by depth 8, the residual lies entirely on variables the prefix logically forces, and classical LP-probing recovers most of them at one LP solve per variable while the network answers all variables in a single 1.6 ms pass — a 19–23× cost advantage at equal or better accuracy. Used as branching guidance this gives a $2.71\times$ median speedup at 21×60 ($p = 0.019$), $6.2\times$ fewer nodes, and a solve rate of 100% against 90.0%; in the deployed cascade, where lattice reduction absorbs a share of instances that falls from 30/30 at $n = 25$ to 16/30 at $n = 60$, guidance lifts the end-to-end rate from 93.3% to 100%. With solution multiplicity matched across sizes, a model trained only at 10×25 is statistically indistinguishable from one trained at the target size, and an earlier “transfer failure” is explained as an ablation artifact.

The role learning plays here is amortization of expensive deduction — the role it plays in branching imitation for mixed-integer programming — and not the discovery of structure that classical reasoning cannot reach. That is a narrower claim than this project set out to make, but it is the one the measurements support, and the measurement procedure that establishes it applies to any planted-solution combinatorial family.

Reproducibility

Code is organized by cycle under `proposed_src/`, with a `ROLES.txt` in each folder describing every file and that cycle’s outcome; `util/` holds modules shared across cycles. Scripts are run from their own directory. The complete cycle reproduces with

```
cd proposed_src/v22_transfer && bash run_v22_cd.sh
```

which builds the $|\mathcal{S}|$ -matched test set, runs conditions C and D under both AHL settings, and emits the cross-arm report. Instance generators, exact-marginal label builders, checkpoints and per-instance result files are enumerated with their paths in Appendix A of `docs/version.md`. The guidance models are `runs/v22/model_n25/conditional.pt` (96 KB) and `runs/v22/model_n50/conditional.pt`; the legacy multi-head network is `runs/best.pth`

(8.1 MB). The repository tracks code and documentation only: weights and instance data are excluded to keep it light, and every such artifact is regenerated from the scripts named in Appendix A. Instances come from `util/generate_hard_instances.py` (training pools, 272 MB) and `v22_transfer/v22_make_testset.py` ($|S|$ -matched test sets); exact-marginal labels from `v15_bayes/v15_make_marginal_labels.py` and `v17_conditional/v17_make_conditional_data.py`; guidance models from `v17_conditional/v17_train_conditional.py`. All generators are seeded, so the regenerated data matches what produced the numbers reported here.

References

- Aardal, K., Hurkens, C. A. J., and Lenstra, A. K. (2000). Solving a system of linear Diophantine equations with lower and upper bounds on the variables. *Mathematics of Operations Research*, 25(3):427–442.
- Aardal, K., Bixby, R. E., Hurkens, C. A. J., Lenstra, A. K., and Smeltink, J. W. (2000). Market split and basis reduction: Towards a solution of the Cornuéjols–Dawande instances. *INFORMS Journal on Computing*, 12(3):192–202.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. (2023). On representing mixed-integer linear programs by graph neural networks. In *International Conference on Learning Representations (ICLR)*.
- Cornuéjols, G. and Dawande, M. (1999). A class of hard small 0-1 programs. In *Integer Programming and Combinatorial Optimization (IPCO)*, pages 284–293.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning

- to branch in mixed integer programming. In *AAAI Conference on Artificial Intelligence*, pages 724–731.
- Lenstra, A. K., Lenstra, H. W., and Lovász, L. (1982). Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534.
- Li, Z., Guo, J., and Si, X. (2023). G4SATBench: Benchmarking and advancing SAT solving with graph neural networks. *Transactions on Machine Learning Research*.
- Marques-Silva, J. P. and Sakallah, K. A. (1999). GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers*, 48(5):506–521.
- Moskewicz, M. W., Madigan, C. F., Zhao, Y., Zhang, L., and Malik, S. (2001). Chaff: Engineering an efficient SAT solver. In *Design Automation Conference (DAC)*, pages 530–535.
- Ohrimenko, O., Stuckey, P. J., and Codish, M. (2009). Propagation via lazy clause generation. *Constraints*, 14(3):357–391.
- Schnorr, C. P. and Euchner, M. (1994). Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1–3):181–199.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2019). Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations (ICLR)*.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. (2018). Graph attention networks. In *International Conference on Learning Representations (ICLR)*.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. (2019). How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*.